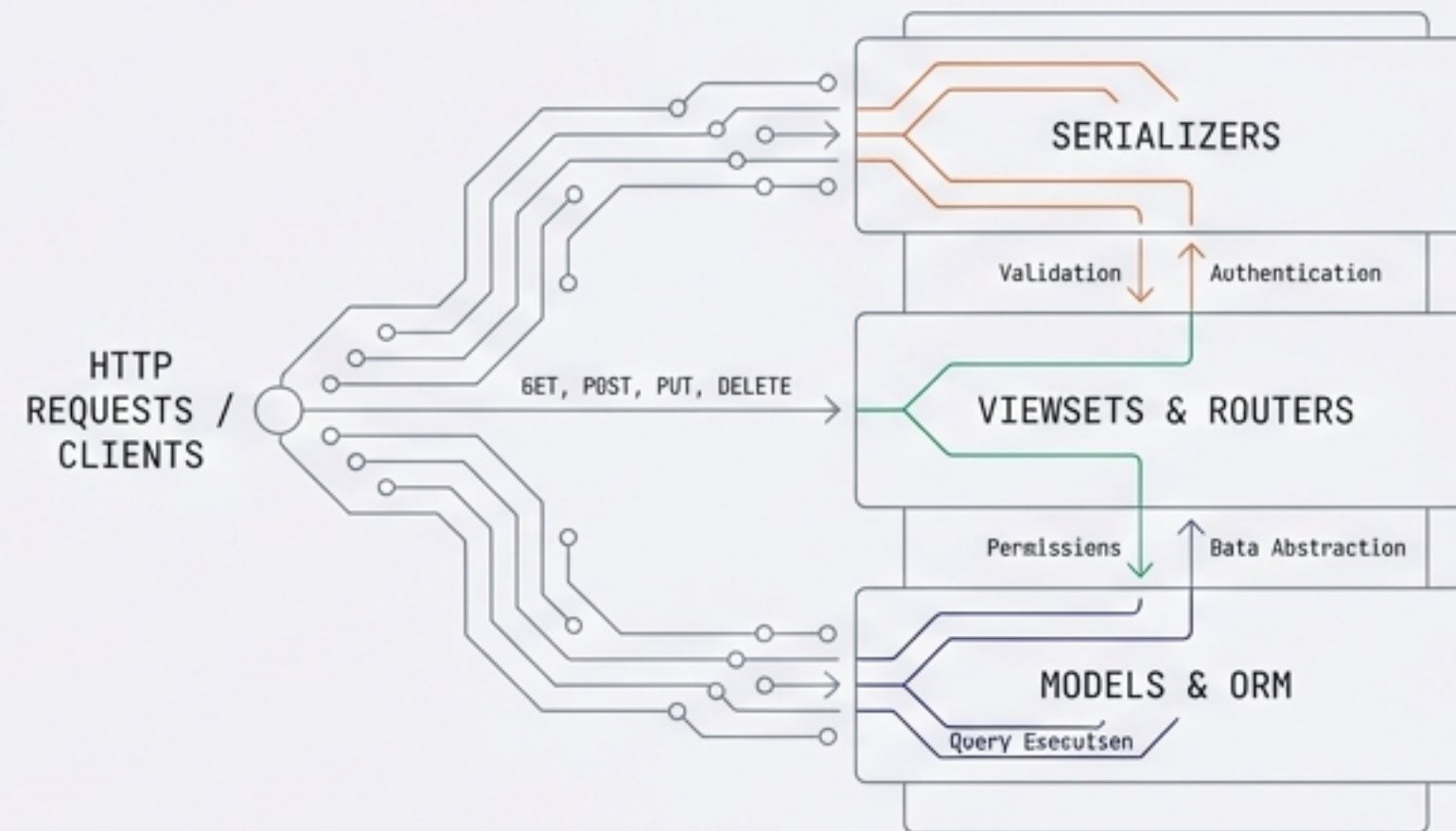


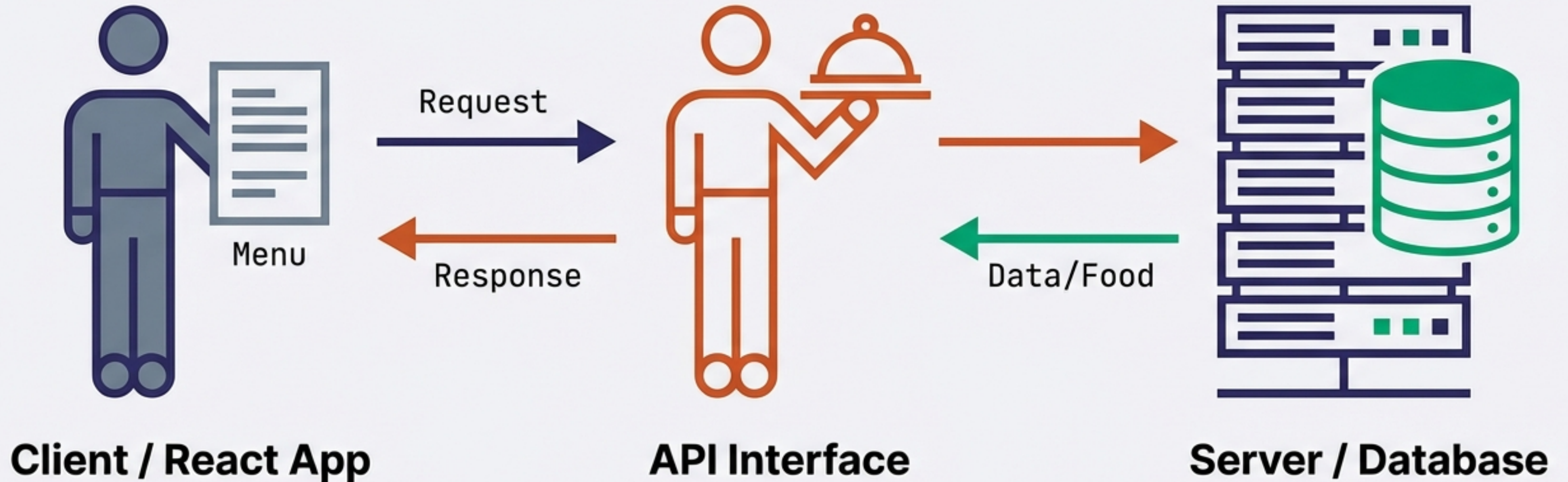
Architectural Foundations of Django REST Framework

From HTTP Concepts to ViewSet Abstractions



A self-paced architectural overview.

The Mental Model: The API as Messenger



The Customer (Client) does not need to know how the food is cooked (Business Logic). They only need to know how to order it from the Menu (Documentation). The API acts as the intermediary.

The Rules of Engagement: What is REST?

REST is an architectural style, not a protocol.

Stateless



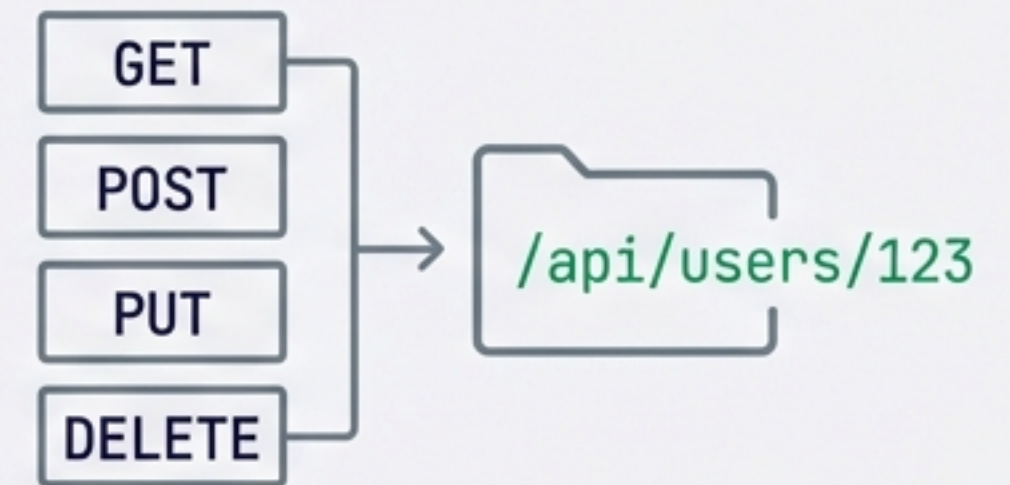
The server has no memory of the conversation. Every request must contain all necessary info, including authentication tokens.

Separation



Frontend and Backend are divorced. You can swap React for Vue without breaking the Django backend.

Uniform Interface



Communication happens via standardized HTTP methods (Verbs) and resources (URIs).

The Vocabulary of HTTP

ACTIONS

GET - Retrieve data (Read)

POST - Create new data (Write)

PUT - Update entire object (Replace)

PATCH - Update partial object (Modify)

DELETE - Remove data

RESULTS

200 OK / 201 Created

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

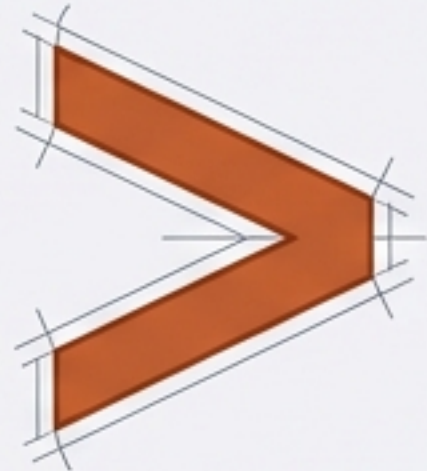
500 Internal Server Error

Why Django REST Framework?



Standard Django
JetBrains Mono

Manual JSON creation.

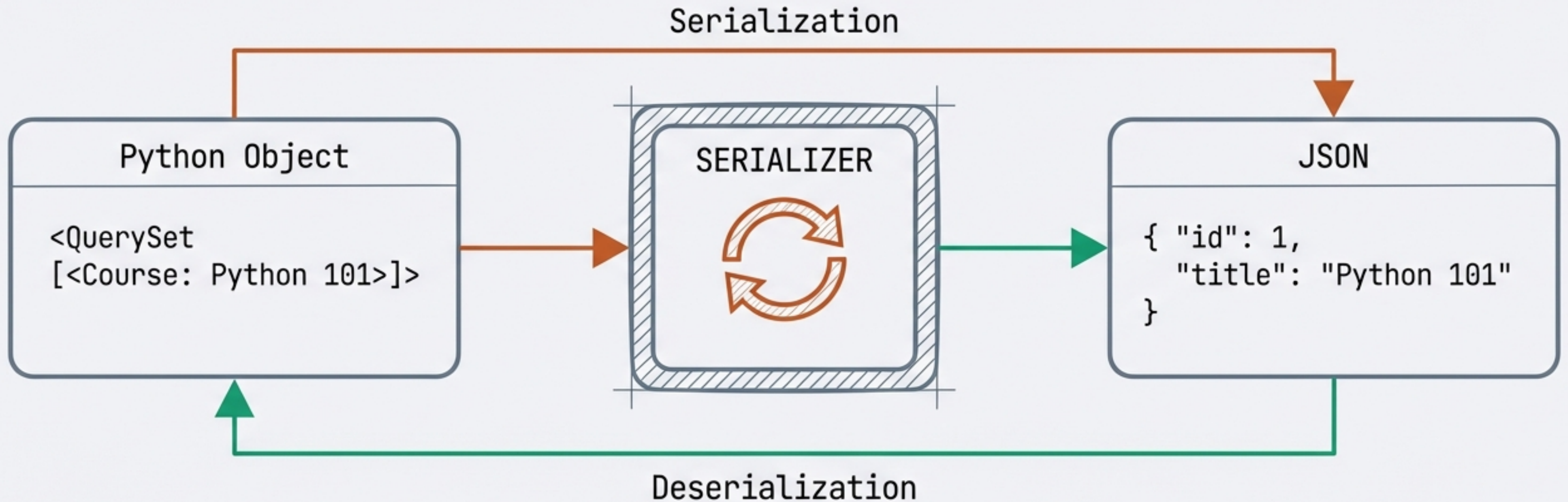


DRF
JetBrains Mono

1. Serialization: QuerySets to JSON automation.
2. Auth & Permissions: OAuth, JWT, Session support.
3. TheBrowsable API



The Translator: Serialization



The internet speaks Text (JSON). Python speaks Objects. The Serializer bridges this gap.

Manual Control vs. Framework Magic

Serializer (Base Class)

```
class ContactSerializer(serializers.Serializer):  
    name = serializers.CharField()  
    email = serializers.EmailField()  
    # Must write .create() manually
```

Best for non-database input (e.g., Login, Email forms).

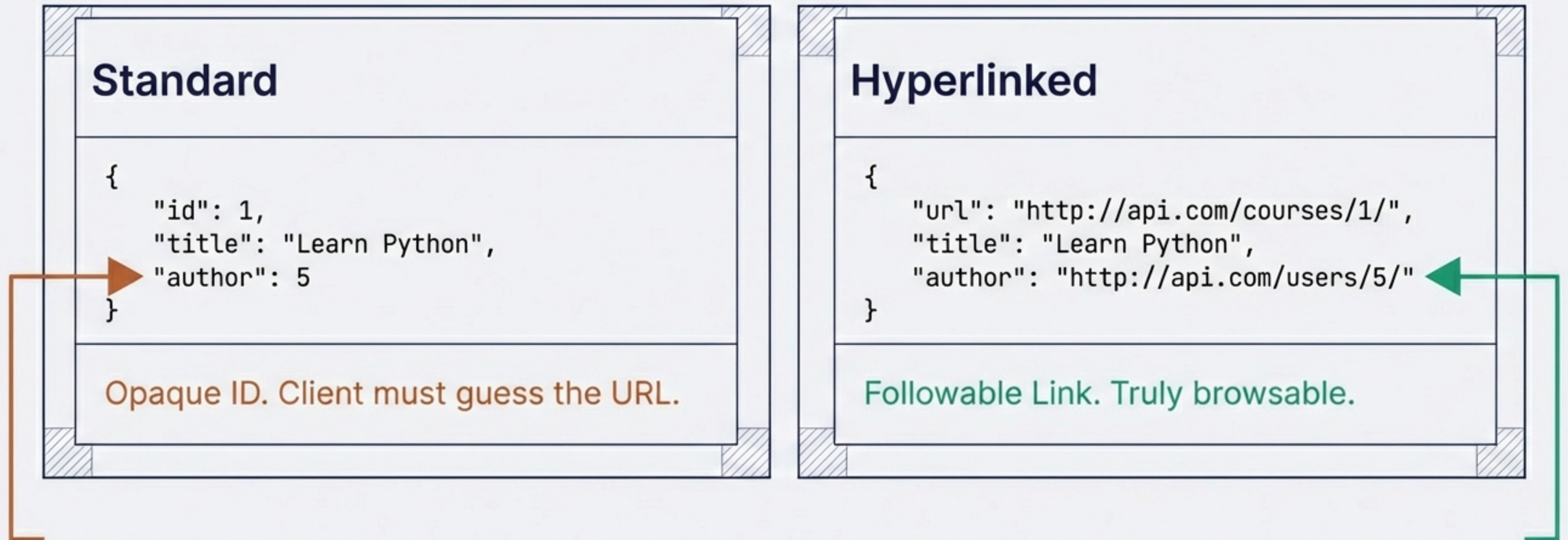
ModelSerializer

```
class CourseSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Course  
        fields = '__all__'
```

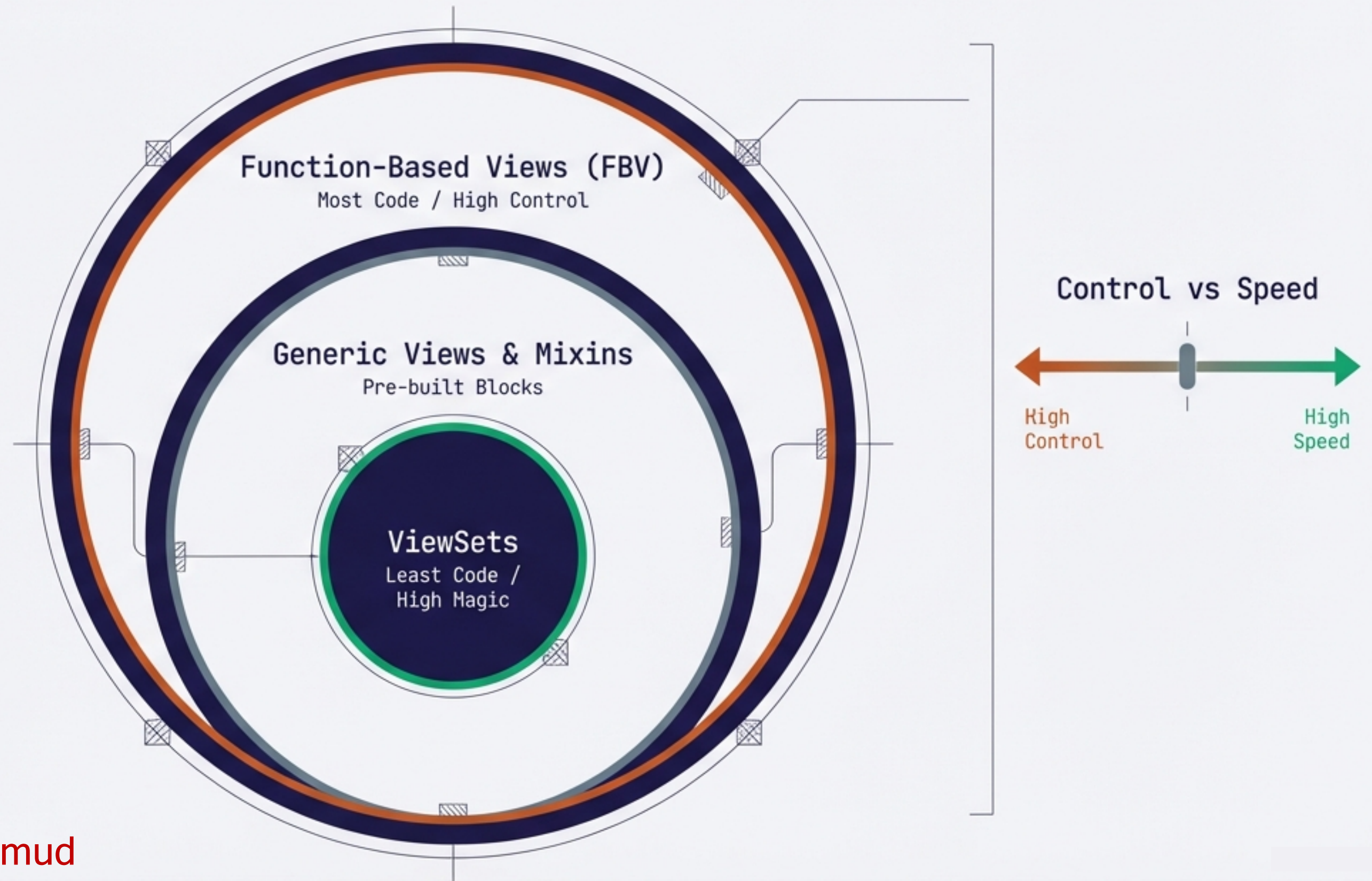
Mirrors the DB Model. Standard for 90% of APIs.

The Web of Data: HyperlinkedModelSerializer

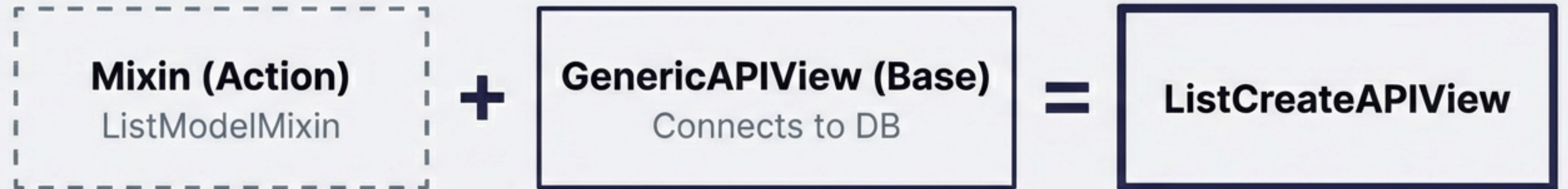
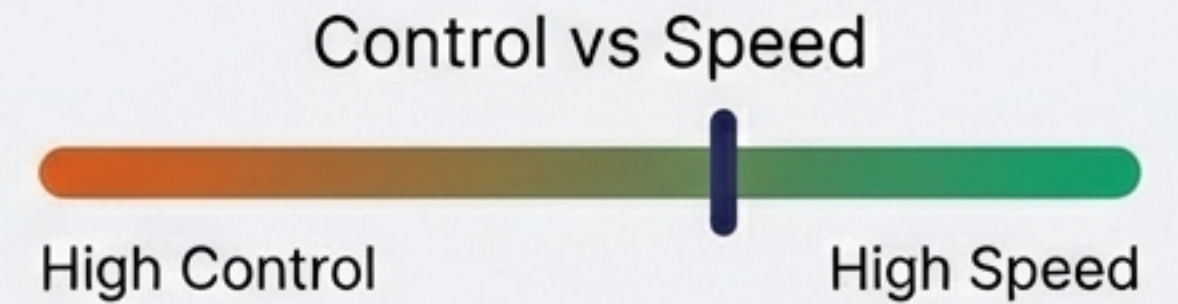
Moving from IDs to URLs (HATEOAS)



The Hierarchy of Abstraction



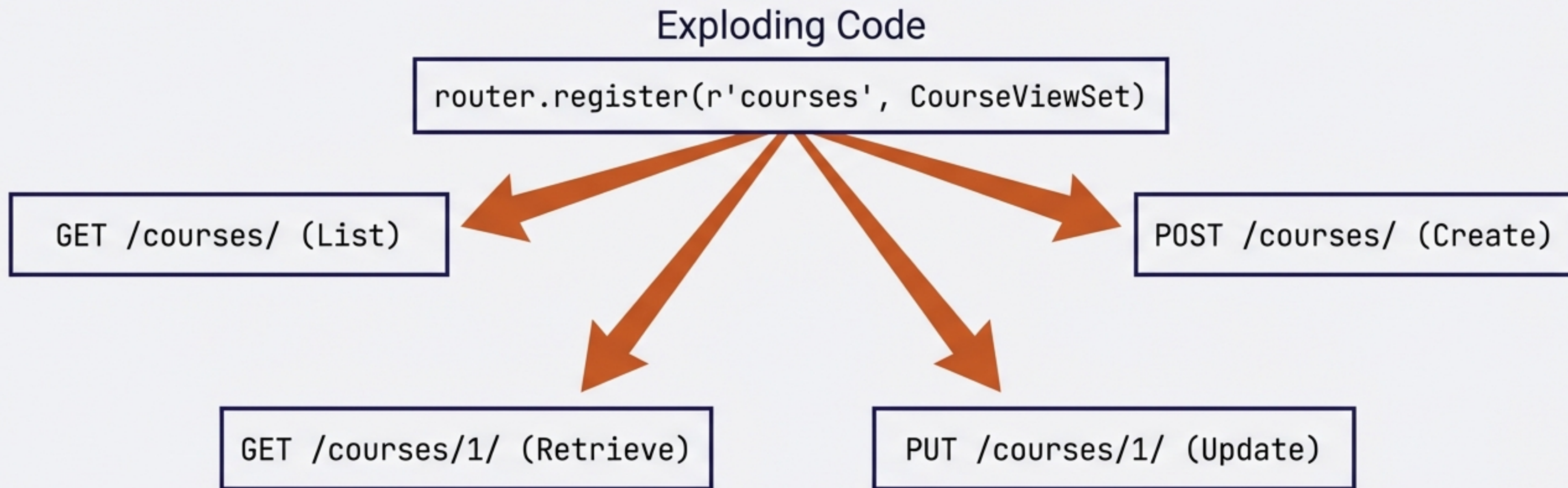
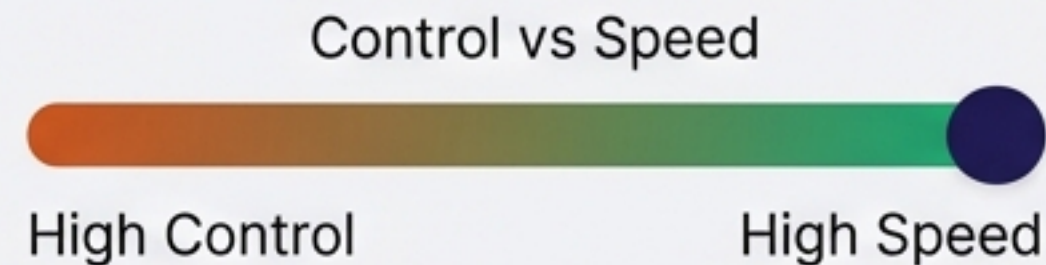
Level 3: Generic Views & Mixins



```
class CourseList(generics.ListCreateAPIView):  
    queryset = Course.objects.all()  
    serializer_class = CourseSerializer
```

Lego Blocks. Standard CRUD where you need to tweak just one thing (like filtering a queryset).

Level 4: ViewSets & Routers



ModelViewSet: Inherits from GenericViewSet + ALL Mixins.
Instant CRUD. Rapid Prototyping.

Strategic Decision Matrix

Choosing the Right Tool for the Job

Tool	Control vs Speed	Best Use Case
@api_view	High Control / Medium Speed	Simple scripts, custom non-DB logic.
APIView	High Control / Medium Speed	Complex integrations (Payments, ML).
Generics	Medium Control / Fast Speed	Standard CRUD with minor tweaks.
ModelViewSet	Low Control / Instant Speed	Standard Database CRUD. Prototyping.